

U.E 5.1 - Intelligence Artificielle : Machine learning

Promotion FISE 2027

ENSTA BRETAGNE
2 Rue François Verny
29200 Brest Cedex 9, France



BARROS ALVES Luiz Felipe
luiz.barros@ensta.fr

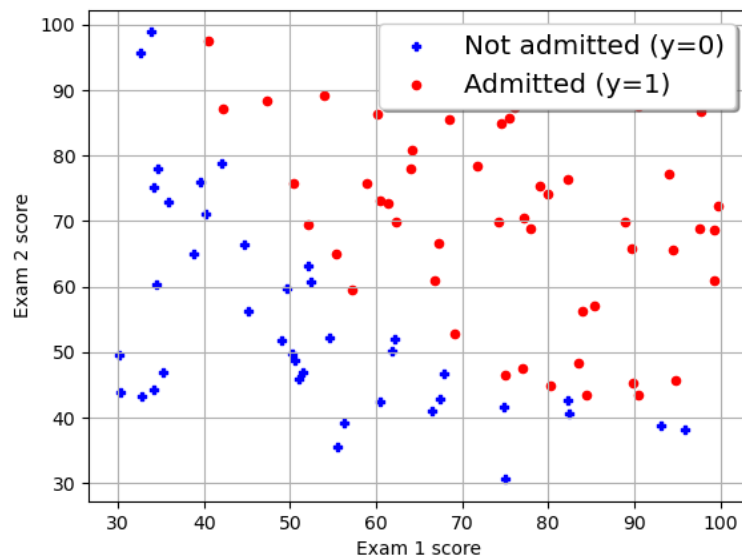
Sommaire

1	Logistic Regression	2
1.1	Visualizing the data	2
1.2	Implementation	3
1.2.1	Sigmoid function	3
1.2.2	Cost function and gradient	4
1.2.3	Learning parameters using fmin tnc	5
1.2.4	Evaluating logistic regression	6

1 Logistic Regression

1.1 Visualizing the data

```
1 import numpy as np
2 from matplotlib import pyplot as plt
3
4
5 def plotData(X,y):
6
7     # ===== YOUR CODE HERE =====
8     # Instructions: Plot data X with different markers
9     # according the value in y
10    # =====
11
12    pos = X[(y==1).flatten(),:]
13    neg = X[(y==0).flatten(),:]
14    plt.figure()
15    plt.scatter(neg[:, 0], neg[:, 1], s=20, marker='P', color=
16                'b')
17    plt.scatter(pos[:, 0], pos[:, 1], s=20, marker='o', color=
18                'r')
19    plt.xlabel('Exam 1 score')
20    plt.ylabel('Exam 2 score')
21    plt.grid(True)
22    plt.legend(['Not admitted (y=0)', 'Admitted (y=1)'], loc='
                upper right', shadow=True, fontsize='x-large', numpoints
                =1)
23    plt.show()
24
25    # =====
```



1.2 Implementation

1.2.1 Sigmoid function

```

1 import numpy as np
2 from matplotlib import pyplot as plt
3
4
5 def plotData(X,y):
6
7     # ===== YOUR CODE HERE =====
8     # Instructions: Plot data X with different markers
9     # according the value in y
10    # =====
11
12    pos = X[(y==1).flatten(),:]
13    neg = X[(y==0).flatten(),:]
14    plt.figure()
15    plt.scatter(neg[:, 0], neg[:, 1], s=20, marker='P', color=
16                'b')
17    plt.scatter(pos[:, 0], pos[:, 1], s=20, marker='o', color=
18                'r')
19    plt.xlabel('Exam 1 score')
20    plt.ylabel('Exam 2 score')
21    plt.grid(True)
22    plt.legend(['Not admitted (y=0)', 'Admitted (y=1)'], loc='
23               upper right', shadow=True, fontsize='x-large', numpoints
24               =1)
25
26    # =====

```

1.2.2 Cost function and gradient

```
1 import numpy as np
2 from sigmoid import sigmoid
3
4 def costFunction(theta, X, y):
5     """ computes the cost of using theta as the
6         parameter for logistic regression."""
7
8     # Initialize some useful values
9     m,n = X.shape    # number of training examples and
        parameters
10    theta = theta.reshape((n,1)) # due to the use of fmin_tnc
11
12
13    # ===== YOUR CODE HERE =====
14    # Instructions: Compute the cost of a particular choice of
        theta.
15
16    #           You should set J to the cost.
17    #
18    arg1 = X @ theta
19    arg2 = sigmoid(arg1)
20    j = (-y*np.log(arg2) - (1-y)*np.log(arg2))
21
22    J = np.sum(j) / m
23
24    # =====
25
26    return J
```

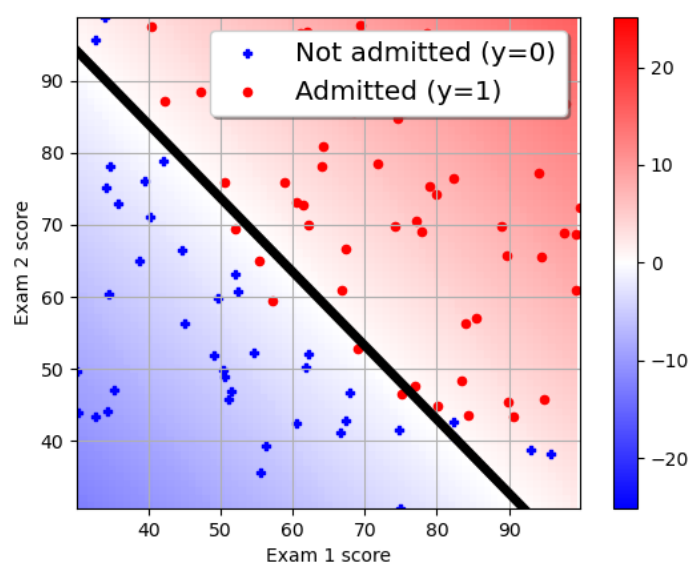
```
1 from sigmoid import sigmoid
2 import numpy as np
3
4
5 def gradientFunction(theta, X, y):
6     """
7         Compute cost and gradient for logistic regression with
            regularization
8
9         computes the cost of using theta as the parameter for
            regularized logistic
10        regression and the gradient of the cost w.r.t. to the
            parameters.
11        """
12
13    # Initialize some useful values
14    # number of training examples
15    m = X.shape[0]
16
```

```

17 # number of parameters
18 n = X.shape[1]
19 theta = theta.reshape((n,1)) # due to the use of fmin_tnc
20
21
22 # gradient variable
23 grad = 0.
24
25
26 # ===== YOUR CODE HERE =====
27 # Instructions: Compute the gradient of a particular
28 #               choice of theta.
29 #               Compute the partial derivatives and set
30 #               grad to the partial
31 #               derivatives of the cost w.r.t. each
32 #               parameter in theta
33
34 arg1 = X @ theta
35 arg2 = sigmoid(arg1)
36 grad = (X.T @ (arg2 - y)) / m
37
38 # =====
39
40 return grad

```

1.2.3 Learning parameters using fmin tnc



The function `plotDecisionBoundary.py` takes the decision at the point where the sigmoid function is equal to 0.5, which is the same as saying that the logit function is equal to 0. Then we have $\theta_0 + \theta_1 x_1 + \theta_2 x_2 = 0$, which is a linear equation in x_1 and x_2 . The function `plotDecisionBoundary.py` takes the two points where the line intersects the axes and plots the line between them.

1.2.4 Evaluating logistic regression

```

1 import numpy as np
2 from sigmoid import sigmoid
3
4
5 def predict(theta, X):
6
7     """ computes the predictions for X using a threshold at
8         0.5
9     (i.e., if sigmoid(theta'*x) >= 0.5, predict 1)
10    """
11
12    # initialisation
13    p = 0
14
15    # ===== YOUR CODE HERE =====
16    # Instructions: Complete the following code to make
17    # predictions using
18    # your learned logistic regression
19    # parameters.
20    # You should set p to a vector of 0's and 1'
21    # s
22
23    p = sigmoid(X @ theta) >= 0.5
24
25    # =====
26
27    return np.atleast_2d(p).T

```

```

Cost at theta found by scipy: 0.203498
Expected cost (approx): 0.203498

-----

theta: ['-25.1613', '0.2062', '0.2015']
Expected theta (approx): -25.1613 0.206 0.201

-----

For a student with scores 45 and 85, we predict an admission probability of 0.776291
Expected Proba (approx): 0.776291

-----

Train Accuracy: 89.000000
Expected accuracy (approx): 89.0%

```